

# Morajaa

Guide Technique & Architecture du Projet

## 1. Les Fondations du Projet

Le projet repose sur trois piliers technologiques distincts. Chacun a un rôle précis et ne doit pas empiéter sur les responsabilités des autres.

### GitHub (Le Coffre-fort)

C'est l'espace de stockage et de versionnage de votre code. Il permet de travailler à deux sans écraser le travail de l'autre grâce au système de "branches".

### Python & FastAPI (Le Moteur)

Le cerveau de l'application (Backend). Il reçoit les demandes, applique la logique métier, interroge la base de données et renvoie la réponse formatée.

### MySQL (La Mémoire)

La base de données où les informations (tuteurs, étudiants) sont stockées de manière permanente et structurée sous forme de tables relationnelles.

## 2. L'Architecture et le Flux Utilisateur

Voici comment les informations circulent lorsqu'un étudiant cherche un tuteur sur la plateforme Morajaa :

 L'Étudiant (Sur son navigateur)



 Front-end (HTML / CSS / JavaScript)

↓ *Envoie une requête API* ↓

 Back-end (Python / FastAPI)

↓ *Exécute une requête SQL* ↓

 Base de Données (MySQL)

## 3. API et JSON : La Langue Commune

**L'API (Le Serveur / Le Serveur) :** C'est le pont de communication. Le Front-end n'a pas le droit de parler directement à la base de données MySQL. Il doit passer par l'API Python.

Pour communiquer, le Python et le Front-end utilisent un format de texte universel appelé le **JSON**. Il ressemble exactement à un dictionnaire Python. C'est ce format que votre base de données devra

## 4. Le Trajet d'une Donnée (Étape par Étape)

---

Voici la traduction concrète du cheminement de l'information entre la base de données et l'écran de l'utilisateur :

1. **Requête SQL (Python vers MySQL) :** Votre code Python demande les données avec une commande type `SELECT * FROM Tuteurs ;`.
2. **Donnée Brute (MySQL vers Python) :** MySQL renvoie un Tuple brut : `( 1 , ' Ahmed ' , ' Rabat ' )`. C'est illisible pour un site web.
3. **Transformation (Python) :** Vous codez une boucle en Python pour transformer ce Tuple en un Dictionnaire lisible.
4. **Traduction Finale :** FastAPI convertit automatiquement le Dictionnaire en texte JSON.
5. **Réponse au Front-end :** Le Front-end reçoit le JSON et l'utilise pour afficher des cartes sur l'écran.

## 5. Le Code Back-end (Ce que vous devez faire)

---

Voici un exemple de ce à quoi ressemble le code de votre API Python avec FastAPI. Ce code crée une "route" web que le front-end pourra appeler.

```
from fastapi import FastAPI app = FastAPI() # Définition de l'adresse web de l'API
@app.get("/tuteurs/maths") def get_tuteurs_maths(): # Simulation des données issues de MySQL
liste_tuteurs = [ {"id": 1, "nom": "Ahmed", "ville": "Rabat", "tarif": 150}, {"id": 2, "nom": "Sara", "ville": "Casablanca", "tarif": 120} ] # FastAPI s'occupe de transformer cette liste en JSON
return liste_tuteurs
```

## 6. Le Code Front-end (Ce que votre partenaire doit faire)

---

Le HTML et le CSS construisent le design de la page, **mais ils sont statiques**. Ils ne savent pas récupérer des données sur un serveur. Pour connecter le design à votre API, votre partenaire devra utiliser **JavaScript**.

Voici comment votre partenaire va insérer du JavaScript dans son HTML pour interroger votre Python :

```
<!-- 1. La zone vide en HTML --> <div id="resultats-recherche"></div> <!--  
2. Le script JavaScript pour appeler Python --> <script> // Appel de l'URL de  
votre API fetch('http://localhost:8000/tuteurs/maths') .then(reponse =>  
reponse.json()) // Traduction en JSON .then(tuteurs => { const conteneur =  
document.getElementById('resultats-recherche'); // Boucle sur les données pour  
créer du HTML tuteurs.forEach(tuteur => { conteneur.innerHTML += ` <div  
class="carte-tuteur"> <h3>${tuteur.nom}</h3> <p>Ville : ${tuteur.ville}</p> </div>  
`; }); }); </script>
```

**CORS (Important) :** Lors des tests sur vos ordinateurs, assurez-vous d'activer le module CORS dans FastAPI, sinon le navigateur bloquera la requête JavaScript pour des raisons de sécurité.

## 7. Méthodologie de Travail en Équipe (Git)

Pour collaborer efficacement sur le même dépôt GitHub sans écraser vos fichiers respectifs, suivez cette discipline :

- **git pull** : À exécuter **avant** chaque session de travail pour récupérer les nouveautés de votre partenaire.
- **git branch [nom\_branche]** : Ne travaillez jamais sur la branche principale (main). Créez une branche (ex: backend-api-recherche).
- **git checkout [nom\_branche]** : Pour vous positionner sur votre espace de travail.
- **git add** . puis **git commit -m "Message"** : Pour sauvegarder localement.
- **git push origin [nom\_branche]** : Pour envoyer votre travail sur GitHub afin qu'il soit testé et fusionné.

### À retenir sur la séparation des rôles :

Votre mission en back-end s'arrête à la livraison d'un flux JSON propre et structuré depuis la base de données vers une URL locale. C'est à votre partenaire front-end de s'occuper de la réception de ce JSON via JavaScript et de son habillage esthétique en HTML/CSS.